

Fiche de Révision Globale : Fouille de Données et Application sur RapidMiner

1.0 Analyse par Règles d'Association : Découvrir les Relations Cachées

Comment découvrir des relations significatives et exploitables au sein de millions de transactions ? L'analyse par règles d'association est une technique fondamentale de la fouille de données qui répond à cette question. Elle vise à extraire des motifs et des corrélations cachées dans de vastes ensembles de données, notamment transactionnelles. Sa maîtrise est essentielle pour des applications comme l'analyse de paniers d'achat, où l'identification de produits fréquemment achetés ensemble permet d'optimiser le placement en rayon, de créer des promotions ciblées et d'améliorer l'expérience client.

1.1 Les Concepts Fondamentaux : Support, Confiance et Lift

Pour évaluer si une règle est statistiquement significative et non le fruit du hasard, il est impératif de maîtriser trois mesures clés. Ces outils nous permettront de passer du bruit statistique à l'information stratégique.

Support

Le support mesure la fréquence d'apparition d'un ensemble d'items dans la totalité des transactions. Il s'agit d'un indicateur de popularité qui répond à la question : "À quelle fréquence cet ensemble d'articles apparaît-il dans les paniers des clients ?". Il est important de noter que le support d'une règle comme $X \& Y \Rightarrow Z$ est identique à celui de $X \& Z \Rightarrow Y$, car il concerne la fréquence de l'ensemble complet $\{X, Y, Z\}$.

Confiance

La confiance est une probabilité conditionnelle qui mesure la fiabilité d'une règle. Pour une règle $X \& Y \rightarrow Z$, la confiance indique la probabilité qu'une transaction contenant déjà le corps de la règle ($\{X, Y\}$) contienne également sa tête (Z). Elle évalue la force de l'inférence. Sa formule est : $\text{Confiance} = \text{Support}(\{X, Y, Z\}) / \text{Support}(\{X, Y\})$

Lift (Corrélation)

Le Lift est une mesure plus robuste qui évalue la pertinence réelle d'une règle en allant au-delà de la simple co-occurrence. Il mesure comment la présence du corps de la règle influence (augmente ou diminue) la probabilité d'observer la tête, comparé à la probabilité de base que cette tête apparaisse seule.

Illustrer avec un Exemple Chiffré

Considérons une base de 4 transactions et un seuil de support minimum de 50%.

ID Transaction	Items
1000	A, C
2000	A, B, C
4000	A, D
5000	B, E, F

Le calcul du support pour les items et leur combinaison est le suivant :

Itemset	Nombre d'occurrences	Support (sur 4 transactions)
{A}	3	75%
{C}	2	50%
{A, C}	2	50%

À partir de ces itemsets fréquents, nous pouvons évaluer deux règles :

1. Règle $A \Rightarrow C$

- **Support** : Le support de {A, C} est de **50%**.
- **Confiance** : Sur les 3 transactions contenant A, 2 contiennent aussi C. La confiance est donc de $2/3$, soit **66.6%**.

2. Règle C => A

- **Support** : Le support de {A, C} est également de **50%**.
- **Confiance** : Sur les 2 transactions contenant C, les 2 contiennent aussi A. La confiance est donc de $2/2$, soit **100%**.

Cet exemple montre que même avec un support identique, la confiance peut varier radicalement selon le sens de la règle.

Analyser les Limites du Support et de la Confiance

Une confiance élevée peut être trompeuse. Considérons le paradoxe suivant, basé sur un groupe de 5000 étudiants :

	Joue au basket	Ne joue pas au basket	Total
Prend des céréales	2000	1750	3750
Ne prend pas de céréales	1000	250	1250
Total	3000	2000	5000

Analysons la règle Jouer au basket => prendre des céréales :

- **Confiance** : $2000 / 3000 = 66.7\%$.

Ce chiffre semble élevé et pourrait inciter à des actions marketing. Cependant, la probabilité de base qu'un étudiant, quel qu'il soit, prenne des céréales est de $3750 / 5000 = 75\%$. Le fait de savoir qu'un étudiant joue au basket *diminue* en réalité la probabilité qu'il prenne des céréales. La règle est donc contre-intuitive et n'apporte aucune valeur.

Démontrer la Puissance du Lift

Pour résoudre ce paradoxe et éviter les conclusions erronées, nous avons besoin d'une métrique plus robuste : le Lift. C'est la solution directe au problème de la confiance trompeuse. Sa formule est : $\text{corr}(A,B) = P(A \wedge B) / (P(A) * P(B))$.

L'interprétation de sa valeur est directe et puissante :

- **Lift = 1** : Indépendance statistique. Les items A et B apparaissent ensemble aussi souvent que le hasard le voudrait. La règle n'apporte aucune information.
- **Lift > 1** : Corrélation positive. La présence de A augmente la probabilité de trouver B. La règle est potentiellement intéressante.
- **Lift < 1** : Corrélation négative. La présence de A diminue la probabilité de trouver B.

Reprenons l'exemple d'une règle X -> Z avec une confiance de 75% et une règle X -> Y avec une confiance de 50%. La première semble meilleure, mais le Lift révèle une autre réalité :

Itemset	Support	Intérêt (Lift)
(X, Y)	25%	2.0
(X, Z)	37,5%	0.9

Ici, le Lift de 2.0 pour (X, Y) indique une forte corrélation positive, rendant cette association très pertinente malgré une confiance plus faible. À l'inverse, le Lift de 0.9 pour (X, Z) montre une légère corrélation négative, suggérant que la règle n'est pas intéressante malgré sa confiance élevée.

L'efficacité de l'extraction de ces règles et de leurs métriques dépend grandement des algorithmes utilisés pour explorer les millions de combinaisons possibles.

1.2 Les Algorithmes d'Extraction

L'extraction des ensembles fréquents (ceux dont le support dépasse un seuil défini) est l'étape la plus coûteuse en calcul. L'efficacité de l'algorithme choisi est donc cruciale pour la faisabilité de l'analyse sur des volumes de données importants.

Évaluer l'Approche Naïve

- **Fonctionnement** : Tester tous les 2^n sous-ensembles d'items possibles en scannant la base de données pour chacun afin de compter ses occurrences.
- **Limite** : Le coût de cette méthode est exorbitant et la rend totalement impraticable pour tout jeu de données de taille réaliste.

Analyser l'Algorithme Apriori

- **Principe Fondamental** : *"Si un ensemble est non fréquent, alors tous ses sur-ensembles sont également non fréquents."* Ce principe est la clé. Il permet d'élaguer massivement l'espace de recherche. Si $\{A\}$ est non fréquent, il est inutile de tester $\{A, B\}$ ou $\{A, B, C\}$.
- **Fonctionnement Itératif** : Apriori procède par niveau, en 3 étapes :
 1. **Génération par niveau** : Il trouve d'abord les itemsets fréquents de taille 1 (L_1), les utilise pour générer les candidats de taille 2 (C_2), et ainsi de suite.
 2. **Étape de jointure** : Pour générer C_k , l'algorithme joint les itemsets fréquents de taille $k-1$ (L_{k-1}) avec eux-mêmes.
 3. **Étape d'élagage** : Avant de scanner la base, tout candidat de C_k dont un sous-ensemble de taille $k-1$ n'est pas dans L_{k-1} est immédiatement supprimé.
- **Problématiques** :
 - **Génération massive de candidats** : Même avec l'élagage, le nombre de candidats peut exploser, restant coûteux en mémoire et en calcul.
 - **Scans multiples de la base de données** : L'algorithme doit scanner l'intégralité de la base à chaque itération ($k+1$ scans pour trouver les k -itemsets).

Analyser en Profondeur l'Algorithme FP-Growth

- **Principe** : FP-Growth surmonte les limites d'Apriori grâce à deux idées clés :
 1. **Compression des données** : Il compresse la base de données dans une structure arborescente compacte, le **FP-tree**, en seulement deux parcours de la base.
 2. **Absence de génération de candidats** : Il extrait les motifs fréquents directement sur cette structure en mémoire, éliminant l'étape coûteuse de génération de candidats.
- **Processus Étape par Étape (Exemple Complet)** Considérons la base de données D de 6 transactions et un $\text{min_support} = 3$. $T_1=\{A,B,D\}$, $T_2=\{B,C,E\}$, $T_3=\{A,B,C,E\}$, $T_4=\{B,E\}$, $T_5=\{A,B,C,E\}$, $T_6=\{A,B,C,D,E\}$
- **Étape 1 : Comptage et Filtrage**
 - Comptage initial : $B(6)$, $E(5)$, $A(4)$, $C(4)$, $D(2)$.
 - L'item D a un support de 2, inférieur à min_support . Il est éliminé.
- **Étape 2 : Ordre Global**

- Les items fréquents sont ordonnés par support décroissant (avec un tie-breaking alphabétique) : **B, E, A, C**.

- **Étape 3 : Nettoyage et Tri des Transactions**

- Chaque transaction est nettoyée (suppression de D) et réordonnée selon l'ordre global :

ID Original	Items nettoyés et triés
T1	{B, A}
T2	{B, E, C}
T3	{B, E, A, C}
T4	{B, E}
T5	{B, E, A, C}
T6	{B, E, A, C}

****Étape 4 : Construction du FP-Tree****

- * Chaque transaction triée est insérée dans l'arbre, en fusionnant les préfixes communs. Le FP-Tree final est :

...

(root)

└ B:6

└└ A:1

└ E:5

└└ C:1

└ A:3

└ C:3

...

****Étape 5 : Extraction pour l'item 'C'****

L'extraction se fait de manière astucieuse en partant des items les moins fréquents (ici 'C'), suivant une approche "diviser-pour-régner" qui travaille sur des sous-problèmes de plus en plus petits.

- * ****Identifier les chemins**** : La ***header table*** permet de trouver tous les chemins menant à un nœud 'C'.

- * ****Construire la base conditionnelle de 'C'**** : C'est la liste des préfixes de ces chemins, avec leurs poids : `{ [B, E] : 1, [B, E, A] : 3 }`.

- * ****Construire le FP-Tree conditionnel pour 'C'**** : En comptant les items dans la base conditionnelle (B:4, E:4, A:3), on construit un nouvel arbre (tous les items restent fréquents car leur support est ≥ 3) :

...

(root_c)

└ B:4

└ E:4

└ A:3

...

* **Lister les motifs fréquents contenant 'C'** : Puisque l'arbre conditionnel est un chemin unique, toutes les combinaisons de ses nœuds, suffixées par 'C', sont des motifs fréquents. Le support de chaque motif est déterminé par le compte minimal de ses items dans l'arbre conditionnel.

* $\{C\}$: support 4 (somme des supports des nœuds 'C' dans l'arbre original)

* $\{A, C\}$: support 3 (le compte de 'A' dans l'arbre conditionnel)

* $\{E, C\}$: support 4 (le compte de 'E')

* $\{B, C\}$: support 4 (le compte de 'B')

* $\{B, E, C\}$: support 4 ($\min(\text{compte}(B), \text{compte}(E)) = 4$)

* $\{B, E, A, C\}$: support 3 ($\min(\text{compte}(B), \text{compte}(E), \text{compte}(A)) = 3$)

Synthétiser la Comparaison

Critère	Apriori	FP-Growth
Principe de base	Approche itérative (par niveau) basée sur l'élagage des sur-ensembles non fréquents.	Compression de la base de données dans une structure arborescente (FP-tree).
Génération de candidats	Oui, génère un grand nombre de candidats à chaque niveau.	Non, aucune génération de candidats n'est nécessaire.
Nombre de scans	Multiple ($k+1$ scans pour trouver les k -itemsets).	Deux scans seulement, quel que soit le nombre de motifs.
Principal avantage	Simplicité conceptuelle et principe facile à comprendre.	Très efficace en mémoire et en temps de calcul, surtout sur des données denses.

La section suivante montre comment appliquer ces concepts dans un outil pratique comme RapidMiner.

1.3 Mise en Pratique sur RapidMiner

Cette section détaille le workflow concret pour implémenter l'analyse des règles d'association dans RapidMiner. Nous nous appuierons sur l'algorithme FP-Growth pour son efficacité.

Étape 1 : Préparation des Données

Les algorithmes d'association ont besoin que tous les articles d'une même transaction soient sur une seule ligne.

- Utilisez l'opérateur Aggregate pour regrouper les données par numéro de facture (Invoice ou N°Ticket) et concaténer tous les produits associés.
- Utilisez ensuite Select Attributes pour ne conserver que la colonne des produits concaténés, et Rename pour lui donner un nom explicite.

Étape 2 : Extraction des Motifs Fréquents

- Insérez l'opérateur FP-Growth.
- Ajustez le paramètre min support. Le réglage de ce seuil est un exercice d'équilibre fondamental : un seuil trop élevé masque des relations potentiellement utiles, tandis qu'un seuil trop bas génère un bruit statistique qui noie les véritables signaux.

- *Conseil d'expert : Commencez avec un support relativement élevé (ex: 0.1) et diminuez-le progressivement. Si vous commencez trop bas sur un grand jeu de données, vous risquez de submerger le système avec des millions de motifs non pertinents.*
- Décochez l'option "find min number of itemsets" pour un contrôle précis basé uniquement sur le seuil de support.

Étape 3 : Génération des Règles

- Ajoutez l'opérateur Create Association Rules à la suite de FP-Growth.
- Définissez le paramètre min confidence (par exemple, 0.5) pour filtrer les règles et ne conserver que les plus fiables.

Étape 4 : Interprétation des Résultats

Le tableau de résultats contient plusieurs colonnes clés : Premises (le corps), Conclusion (la tête), Confidence et Lift.

- **Lift supérieur à 1** : L'association est positivement corrélée. La présence des prémisses favorise la présence de la conclusion. C'est une règle potentiellement intéressante et exploitable.
- **Confiance de 1 (100%)** : Cette valeur indique une certitude. Dans 100% des cas où les produits des prémisses sont présents, le produit de la conclusion l'est aussi.

2.0 La Classification : Prédire des Catégories

Comment automatiser la prise de décision en attribuant des catégories à de nouvelles données ? La classification, une technique d'apprentissage automatique supervisé, répond à ce besoin. Son objectif est d'attribuer une classe ou une catégorie prédéfinie à de nouvelles observations. En apprenant à partir de données historiques étiquetées, elle permet de construire des modèles prédictifs qui jouent un rôle central dans des domaines allant de la détection de fraude à la segmentation marketing.

2.1 Les Fondamentaux de la Classification

Distinguer Classification et Prédiction

Critère	Classification	Prédiction
Définition	Attribuer une donnée à une classe prédéfinie.	Estimer une valeur numérique continue.
Sortie Attendue	Une étiquette (catégorie qualitative).	Une valeur numérique (quantitative).
Exemple	Prédire si un email est un spam ou non.	Prédire le chiffre d'affaires du mois prochain.

Décrire le Processus de Construction d'un Modèle

1. **Étape 1 : Construction du Modèle** Cette phase utilise un **jeu d'apprentissage** (*training set*), où chaque exemple est déjà associé à une classe connue (le label). L'algorithme analyse ces données pour identifier des schémas et construire un modèle, qui peut prendre la forme de règles (SI-ALORS) ou d'un arbre de décision.
2. **Étape 2 : Estimation de la Performance** Le modèle est ensuite évalué sur un **jeu de test** inconnu pour mesurer sa capacité à généraliser. Son **accuracy** (exactitude) est calculée en comparant ses prédictions aux labels réels, ce qui correspond au pourcentage de tuples correctement classifiés.

Situer l'Apprentissage Supervisé vs. Non Supervisé

- **Apprentissage Supervisé** : Les données d'entraînement sont étiquetées. L'objectif est d'entraîner un modèle capable de prédire correctement le label pour de nouvelles données. La **classification** en est l'exemple par excellence.
- **Apprentissage Non Supervisé** : Les labels des données sont inconnus. L'objectif est de découvrir des structures ou des groupes inhérents aux données elles-mêmes (regroupement ou **clustering**).

Parmi les algorithmes de classification, les arbres de décision sont l'un des plus populaires et des plus intuitifs.

2.2 Analyse Approfondie des Arbres de Décision

L'arbre de décision est un outil qui modélise une série de décisions sous une forme arborescente, imitant le raisonnement humain. Sa principale force réside dans son interprétabilité : il transforme des règles complexes en un diagramme visuel et facile à comprendre.

Décrire l'Anatomie d'un Arbre

Un arbre est composé de trois éléments fondamentaux :

- **Nœuds internes** : Représentent les tests effectués sur les attributs (ex: "L'âge est-il inférieur à 30 ?").
- **Arcs** : Correspondent aux résultats possibles des tests (ex: "Oui" ou "Non").
- **Nœuds feuilles** : Représentent les classes finales prédites par le modèle (ex: "Achète l'ordinateur").

Expliquer le Processus de Construction

La construction suit une approche "top-down récursive" :

1. **Initialisation** : Toutes les données sont regroupées à la racine.
2. **Pré-requis des attributs** : Les attributs numériques sont souvent discrétisés (transformés en catégories).
3. **Partitionnement récursif** : À chaque nœud, les données sont séparées en sous-ensembles en fonction de la valeur d'un attribut de test.
4. **Choix de l'attribut** : Le choix de l'attribut de test est crucial. Il est guidé par des heuristiques, comme le gain informationnel, pour maximiser la "pureté" des sous-ensembles.
5. **Conditions d'arrêt** : Le processus s'arrête lorsqu'un nœud ne contient que des tuples de la même classe (il devient une feuille).

Analyser l'Importance du Choix de l'Attribut

Le choix de l'attribut de partitionnement a un impact direct sur la simplicité de l'arbre. Considérons le jeu de données suivant :

A	B	Classe
0	1	C1
0	0	C1
1	1	C2
1	0	C2

- **Cas 1 (Partitionnement avec A)** : L'arbre est simple et efficace. Si A=0, la classe est C1. Si A=1, la classe est C2. L'arbre est construit en une seule étape.
- **Cas 2 (Partitionnement avec B)** : L'arbre est plus complexe. Les branches B=0 et B=1 contiennent toutes deux un mélange des classes C1 et C2, nécessitant un second niveau de partitionnement sur l'attribut A.

Introduire la Notion d'Entropie

Pour choisir l'attribut optimal de manière quantitative, les algorithmes utilisent l'**entropie**, issue de la théorie de l'information. L'entropie mesure l'incertitude ou l'impureté d'un ensemble de données. Un ensemble pur a une entropie de 0. L'objectif est de choisir l'attribut qui réduit le plus l'entropie, c'est-à-dire qui apporte le plus d'information. Comme l'illustre l'analogie du temps dans le désert, apprendre qu'il fera beau demain est une information de faible valeur, car c'est très probable. En revanche, apprendre qu'il va pleuvoir est une information de

grande valeur. De même, un attribut qui sépare nettement les classes apporte beaucoup d'information et est donc un bon candidat pour le partitionnement. Pour un ensemble contenant p exemples positifs et n exemples négatifs, l'entropie se calcule ainsi : $I(p, n) = -p/(p+n) \log_2(p/(p+n)) - n/(p+n) \log_2(n/(p+n))$.

Gérer le Surapprentissage (Overfitting)

- **Définition** : Le surapprentissage (overfitting) se produit lorsqu'un modèle mémorise le bruit et les anomalies du jeu d'entraînement au lieu des tendances générales. Sa performance est excellente sur les données d'entraînement mais s'effondre sur de nouvelles données.
- **Stratégies de Pruning (Élagage)** : Pour éviter cela, on simplifie l'arbre via l'élagage.

Approche	Méthode	Avantage	Inconvénient
Prepruning	Arrêter la construction de l'arbre prématurément si un nouveau découpage n'apporte pas un gain suffisant.	Rapide, l'arbre est directement plus simple.	Le choix du seuil est difficile et peut stopper la construction trop tôt.
Postpruning	Construire l'arbre complet, puis supprimer les branches qui n'améliorent pas la performance sur un jeu de validation.	L'arbre final est souvent plus robuste.	Le processus est plus long car il nécessite la construction complète.

Extraction des Règles de Classification

L'un des principaux avantages des arbres de décision est la facilité avec laquelle on peut en extraire des règles lisibles sous la forme SI-ALORS (IF-THEN). Chaque chemin unique de la racine à une feuille constitue une règle. Les conditions des nœuds internes le long du chemin forment la partie SI, et la classe de la feuille constitue la partie ALORS.

2.3 Au-delà des Arbres de Décision : Autres Classifieurs

Bien que les arbres de décision soient un excellent point de départ, de nombreuses autres familles d'algorithmes de classification existent, offrant souvent des performances supérieures.

La Classification Bayésienne

Cette approche probabiliste est fondée sur le théorème de Bayes. Pour une nouvelle donnée, le modèle calcule la probabilité de son appartenance à chaque classe possible et lui attribue la classe ayant la probabilité la plus élevée. Les classifieurs bayésiens, notamment le "naïf bayésien", servent souvent de référence pour évaluer la performance d'autres méthodes.

Les Forêts Aléatoires (Random Forest)

Une forêt aléatoire est une méthode d'ensemble qui combine plusieurs arbres de décision. Chacun est entraîné sur un sous-ensemble légèrement différent des données. Pour classer une nouvelle donnée, chaque arbre de la forêt "vote", et la classe majoritaire l'emporte. Cette approche collaborative augmente la précision et réduit significativement le risque de surapprentissage par rapport à un arbre unique.

2.4 Mise en Pratique sur RapidMiner (Cas du Titanic)

Cette section applique les concepts de classification pour construire un modèle capable de prédire la survie des passagers du Titanic.

Étape 1 : Construire un Premier Arbre

- Utilisez l'opérateur Set Role pour désigner l'attribut "Survived" comme le Label à prédire.
- Appliquez l'opérateur Decision Tree. L'arbre généré identifie immédiatement l'attribut "Sex" comme le facteur le plus discriminant.

Étape 2 : Simplifier et Interpréter

- Pour rendre l'arbre plus lisible, réduisez le paramètre maximal depth (par exemple, à 4 ou 2). Cette action simplifie radicalement sa structure en ne conservant que les règles de décision les plus importantes.

Étape 3 : Évaluer la Performance

- Utilisez Split pour diviser les données (par exemple, 80% pour l'apprentissage, 20% pour le test).
- Utilisez Apply Model pour appliquer l'arbre appris sur le jeu de test et générer des prédictions.
- Utilisez Performance (Classification) pour comparer les prédictions aux véritables labels. Cet opérateur génère des métriques clés comme Class Recall (capacité à trouver les vrais positifs) et Class Precision (fiabilité des prédictions positives).

Étape 4 : Analyser le Compromis Complexité vs. Précision

- La conclusion est qu'un arbre plus complexe (avec une profondeur plus grande) est souvent plus efficace en termes de précision, même s'il est moins facile à interpréter. Le choix final dépend de l'objectif : interprétabilité maximale ou performance prédictive maximale.

Étape 5 : Comparer avec une Forêt Aléatoire

- Remplacez l'opérateur Decision Tree par Random Forest. En théorie, ce modèle d'ensemble, qui combine plusieurs arbres de décision, devrait offrir une meilleure précision globale en réduisant le risque de surapprentissage.

3.0 Le Clustering : Identifier des Groupes Naturels

Comment découvrir des groupes naturels et cachés dans nos données lorsque nous ne connaissons pas les catégories à l'avance ? Le clustering est une technique d'analyse non supervisée qui répond à ce défi. Son objectif est de regrouper des données en sous-ensembles homogènes (clusters) en se basant sur le principe de similarité : maximiser la ressemblance intra-groupe et minimiser la ressemblance inter-groupe.

3.1 La Mesure de la Similarité : Le Cœur du Clustering

La quantification de la similarité via une mesure de distance est l'étape la plus critique du processus. Un choix inapproprié peut mener à des regroupements dénués de sens. Le choix de la métrique doit être scrupuleusement adapté aux types de variables présentes.

Analyser le Traitement des Variables Numériques

- **Standardisation** : Lorsque les variables ont des échelles très différentes (ex: âge et salaire), la variable à grande échelle dominera le calcul de distance, rendant la contribution des autres négligeable. La standardisation est donc indispensable pour un traitement équitable.
- **Calcul du Z-score** : C'est une méthode de standardisation courante qui se déroule en 3 étapes :
 1. Calculer la moyenne (mf).
 2. Calculer l'écart absolu moyen (sf).
 3. Appliquer la formule pour chaque valeur : $z = (x_{if} - mf) / sf$.
- **Démonstration par l'Exemple** : Considérons trois personnes (p1, p2, p3) avec leurs données brutes et standardisées, et calculons la distance de Manhattan.

	Données Brutes	Données Standardisées
	Âge	Salaire
p1	50	11000
p2	70	11100

p3	60	11122
----	----	-------

1. **Avant standardisation :**

- $d(p1, p2) = |50-70| + |11000-11100| = 20 + 100 = 120$
- $d(p1, p3) = |50-60| + |11000-11122| = 10 + 122 = 132$
- Conclusion erronée : p1 est plus proche de p2.

2. **Après standardisation :**

- $d(p1, p2) = |-2.0 - 2.0| + |-2.0 - 0.70| = 4.0 + 2.7 = 6.7$
- $d(p1, p3) = |-2.0 - 0.0| + |-2.0 - 1.30| = 2.0 + 3.3 = 5.3$
- Conclusion correcte : p1 est plus proche de p3.

La standardisation a inversé la conclusion, produisant un résultat statistiquement fiable.

Détailler les Métriques de Distance

- La **distance de Minkowski** est une formule généralisée : $d(i,j) = (|x_{i1} - x_{j1}|^q + \dots + |x_{ip} - x_{jp}|^q)^{1/q}$.
- Ses deux cas d'usage les plus fréquents sont :
 - **Distance de Manhattan (q=1)** : Additionne les écarts absolus.
 - **Distance Euclidienne (q=2)** : Correspond à la distance "à vol d'oiseau".

Synthétiser le Traitement des Autres Variables

Type de Variable	Méthodes et Métriques
Binaires	Pour les variables symétriques (ex: Sexe), on utilise le coefficient d'appariement simple . Pour les asymétriques (ex: test HIV positif), on utilise le coefficient de Jaccard , qui ignore les correspondances 0-0.
Nominales	Variables catégorielles sans ordre (ex: couleur). On peut utiliser le matching simple (rapport des non-correspondances) ou transformer chaque modalité en une nouvelle variable binaire.
Ordinales	Variables avec un ordre (ex: S, M, L). La méthode consiste à remplacer la valeur par son rang, puis à normaliser ce rang entre [0, 1].

Les algorithmes suivants utilisent ces mesures de distance pour former des groupes.

3.2 Panorama et Comparaison des Algorithmes

Il existe trois grandes familles d'algorithmes de clustering : le partitionnement, les algorithmes hiérarchiques et ceux basés sur la densité.

Analyser K-Means (Partitionnement)

- **Principe** : Représenter chaque cluster par un point central (barycentre) et affecter chaque objet au cluster dont le centre est le plus proche.
- **Fonctionnement Itératif** :
 1. **Affectation** : Chaque objet est affecté au centre de cluster le plus proche.
 2. **Mise à jour** : Le centre de chaque cluster est recalculé comme la moyenne de tous les objets qui lui sont affectés. Le processus se répète jusqu'à stabilisation.

- **Forces et Faiblesses :**
 - **Forces :** Simple, efficace sur de grands jeux de données.
 - **Faiblesses :** Le nombre k doit être spécifié, sensible aux outliers, ne fonctionne qu'avec des données numériques, et ne peut découvrir que des formes de clusters convexes (sphériques).

Analyser DBSCAN (Basé sur la Densité)

- **Principe :** Définit les clusters comme des régions denses de l'espace, séparées par des régions de faible densité. Les points isolés sont considérés comme du bruit.
- **Paramètres clés :**
 - **Eps :** Le rayon du voisinage autour d'un point.
 - **MinPts :** Le nombre minimum de points devant se trouver dans ce voisinage.
- **Avantages Majeurs :** Sa capacité à trouver des groupes de formes arbitraires (non-convexes) et sa robustesse face au bruit, qu'il identifie et isole.

Présenter le Clustering Hiérarchique

Cette approche crée une décomposition hiérarchique des objets, visualisée sous la forme d'un dendrogramme. Il existe deux approches :

- **Agglomérative (AGNES) :** Ascendante (bottom-up), fusionne progressivement les clusters.
- **Divisive (DIANA) :** Descendante (top-down), divise progressivement le cluster initial.

Comparer les Algorithmes en Pratique

- **Impact de la Normalisation sur K-Means :** Bien que les résultats sur données brutes puissent sembler corrects, la normalisation est une étape statistiquement **nécessaire** pour obtenir des résultats fiables et éviter que les variables à grande échelle ne dominent le processus.
- **Gestion du Bruit : K-Means vs. DBSCAN :** Face à un point aberrant (ex: (10,5)), K-Means déforme fortement l'un des clusters pour l'intégrer, compromettant le regroupement. En revanche, DBSCAN identifie correctement ce point comme du bruit et le laisse de côté, préservant ainsi l'intégrité des clusters existants.

En synthèse, alors que K-Means est simple et rapide, sa fragilité face aux outliers et son exigence de formes de données convexes rendent DBSCAN un choix souvent plus robuste pour les jeux de données du monde réel, qui sont rarement propres ou parfaitement sphériques.

Tableau Comparatif Final

Critère	K-Means	DBSCAN
Forme des clusters	Découvre uniquement des groupes convexes (sphériques).	Peut découvrir des groupes de formes arbitraires (non-convexes).
Paramètres requis	Le nombre de clusters k doit être spécifié à l'avance.	Nécessite de spécifier Eps (rayon) et MinPts (densité).
Gestion des outliers	Très sensible ; les outliers peuvent fortement influencer les centres.	Robuste ; identifie et isole les outliers comme du "bruit".
Type de données	Applicable principalement aux données numériques.	Applicable tant qu'une mesure de distance peut être définie.